Boids - a walkthrough

Boids

Three rules, two and a half thousand birds, and a murmuration that no line of code describes.

Steer away from whoever is too close. Steer toward the average heading of whoever is near. Steer toward the average position of whoever is near. That is the entire model.

> *No bird knows it is in a flock, and no bird is told to stay near the others — the murmuration is what those three local rules look like from far enough away.*

The shortest possible summary

This is the third of three GPU demos, and the commit describes the arc across them in one sentence:

> *grayscott is a field talking to itself, physarum is agents talking through a shared field, this is agents talking to each other directly, no field between them.*

That difference is not a detail of style. It forces a correctness discipline the other two did not need. A boid's update reads **every other boid's** position and velocity, so updating in place would let boid 400's new position leak into boid 50's read of it, purely by scheduling luck — and the result would still look like a flock, and would differ on every run.

So position and velocity are ping-ponged: Gray-Scott's `u` / `v` pair applied to a flock, so that every boid this frame sees the **same frozen instant** of everyone else, by construction rather than by hope.

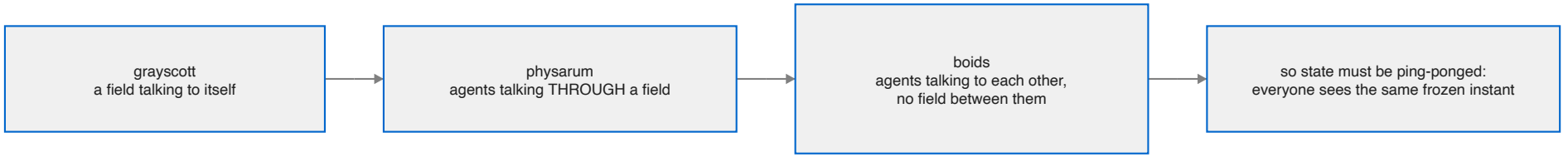
The neighbour search is brute force, every boid against every other, and that is a deliberate choice with numbers behind it — see chapter 3.

These documents

Chapter	What it covers
1. Reynolds' three rules	1987, and why boids became the standard example of emergence
2. The rules, in code	Separation, alignment, cohesion — and the wrapped distance
3. Why this belongs on a GPU	O(n²) on purpose, what it measured, and the ping-pong
4. The five kernels	Including the one that runs backwards, and a missing `atan2f`
5. A frame, end to end	Four dispatches, an explicit parity, and decay in place
6. What to understand	What will surprise you, and what will bite

At a glance

Source	`examples/boids/main.mojo`, 819 lines
Boids	2,500 — a visual choice, not a ceiling
Neighbour search	brute force, every pair, every frame
Per frame	4 dispatches: boids, splat, decay, colour



1. Reynolds' three rules

1987

Craig Reynolds presented **"Flocks, Herds, and Schools: A Distributed Behavioral Model"** at SIGGRAPH in 1987. He called his simulated creatures *boids* — bird-oid objects — and the paper is one of the most-cited in computer graphics, for a reason that has little to do with birds.

Animating a flock had, until then, been understood as an animation problem: somebody had to specify the path of the flock. Reynolds' claim was that this is the wrong level to work at. Give each creature a small set of rules about its immediate neighbours, run them all at once, and flocking is not something you specify — it is something that **happens**.

Three rules, and he was explicit that they are ordered by priority:

1. **Separation** — steer to avoid crowding local flockmates.
2. **Alignment** — steer toward the average heading of local flockmates.
3. **Cohesion** — steer toward the average position of local flockmates.

Local is doing heavy lifting in all three. A boid perceives only what is within a radius of it. Nothing has access to the flock as an object, because there is no flock as an object — only birds with neighbours.

The example states the same thing in its header:

> *No bird knows it is in a flock, and no bird is told to stay near the others.*

Why it mattered beyond graphics

Boids became the canonical demonstration of **emergence**: complex global behaviour arising from simple local rules, with no central controller and no representation of the global pattern anywhere in the system.

It is the same claim Gray-Scott makes about two chemicals and Physarum makes about stigmergy, and boids is the one that made the argument famous — partly because it is so easy to verify by looking. You can watch a flock split around an obstacle and rejoin, and know that nothing anywhere decided to do that.

The model reached film quickly. The bat swarms and penguin herds in *Batman Returns* (1992) were animated with boids-derived software, and Reynolds received a Scientific and Engineering Academy Award in 1998 for the work.

The three rules are in tension, and that is the point

A model where the rules agreed would be uninteresting. These pull against each other:

- **Cohesion** pulls boids together.
- **Separation** pushes them apart.
- **Alignment** does neither, and instead makes them *agree* — which turns a crowd into a flock rather than a cluster.

Flocking is what the equilibrium looks like. Change the balance and you get a different creature, which is exactly what the four presets do:

preset	align	cohesion	separation	radius
flock	0.06	0.0020	0.9	46
swarm	0.02	0.0060	0.5	70
school	0.10	0.0035	1.4	34
scatter	0.01	0.0006	2.2	46

Read them as characters:

- **swarm** — weak alignment, strong cohesion, weak separation, wide radius: everyone wants to be in the middle and nobody cares which way they face. A dense milling ball, like insects.
- **school** — strong alignment, strong separation, *narrow* radius: everyone matches heading with a few close neighbours while keeping distance. Tight ordered ranks, like fish.

- **scatter** — almost no alignment or cohesion, dominant separation: the rules that make a flock are switched off and only the one that breaks it remains.

And the file is upfront about where these numbers come from:

```
# Like Physarum and unlike Gray-Scott, there is no narrow sweet spot here
# either -- these four are this file's own tuning, named for the character
# they give the flock rather than looked up from a source.
```

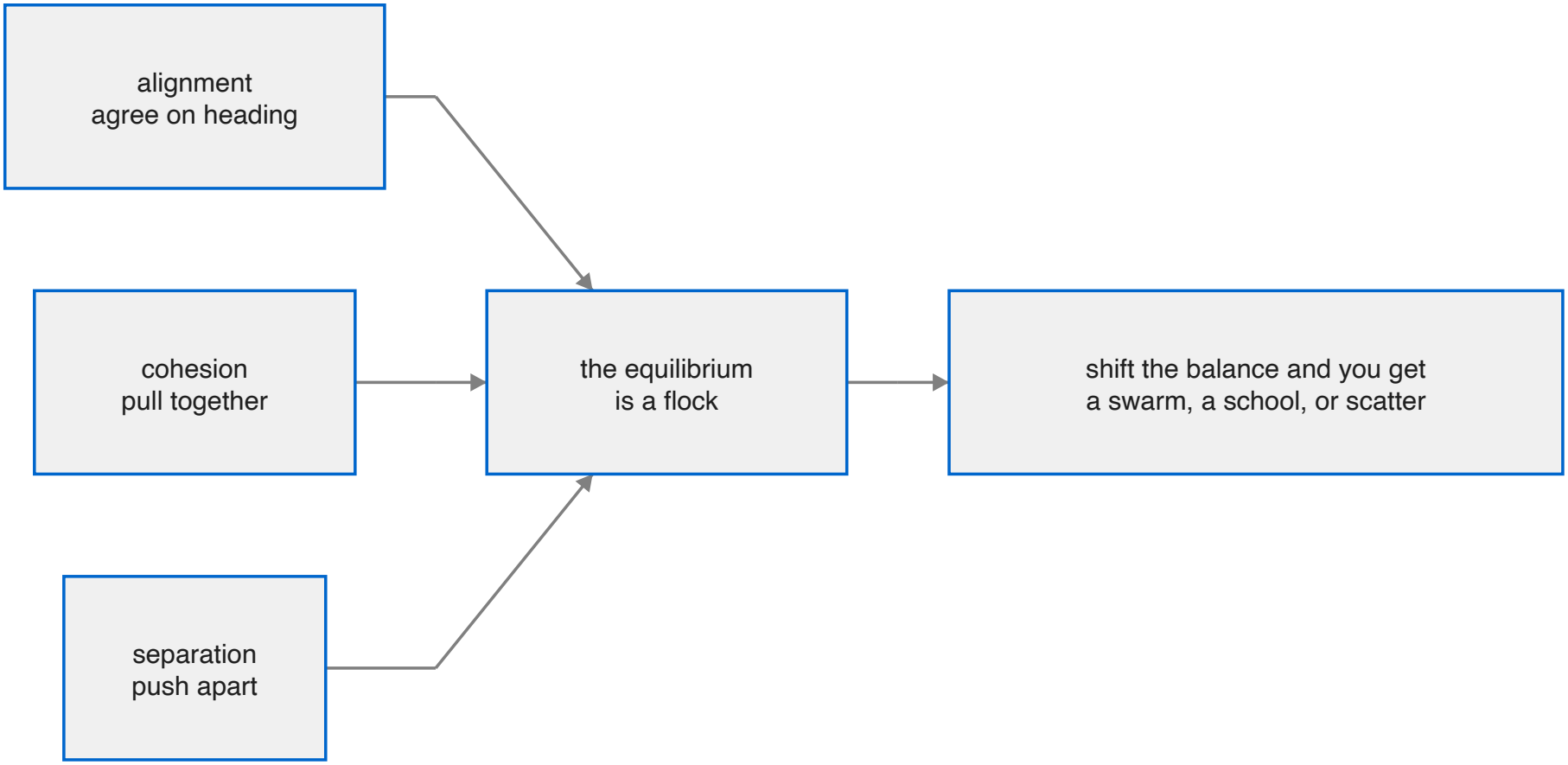
That is the third time this collection has stopped to say which kind of numbers it is using. Gray-Scott's are cited because its parameter plane has sharp boundaries and a published map; Physarum's and Boids' are described as tuning. Getting that distinction into the source is a small discipline that pays every time somebody wonders whether a constant is load-bearing.

Speed limits, which Reynolds also needed

```
comptime MAX_SPEED = Float32(3.4)
comptime MIN_SPEED = Float32(1.1)
```

Not decoration. The three rules are all *accelerations*, and accelerations accumulate — without a cap, boids in a strong flock keep speeding each other up. And without a floor, a boid whose forces happen to cancel stops dead and becomes a permanent obstacle rather than a bird.

A real bird has a stall speed and a maximum. Two clamps, and the flock keeps moving.


alignment
agree on heading
cohesion
pull together
separation
push apart
the equilibrium
is a flock
shift the balance and you get
a swarm, a school, or scatter

2. The rules, in code

A boid is four floats: `px`, `py`, `vx`, `vy`. Four separate arrays — structure of arrays, for the coalescing reason chapter 3 covers.

The scan

```
for j in range(BOIDS):
    if j == i:
        continue
    var dx = px_in[unsafe_offset=j] - x
    var dy = py_in[unsafe_offset=j] - y
```

Every boid, every other boid, every frame. What happens inside decides which rule each neighbour contributes to.

The wrapped distance, which is easy to get wrong

```
# The shortest way around a WRAPPED field, or a neighbour just
# across the seam looks like it is on the far side of the
# screen, and the flock tears itself apart at the edges.
if dx > Float32(WIDTH) * Float32(0.5):
    dx -= Float32(WIDTH)
elif dx < -Float32(WIDTH) * Float32(0.5):
    dx += Float32(WIDTH)
```

The world is a torus — a boid leaving the right edge reappears on the left. So two boids five pixels apart across the seam are at x = 766 and x = 3, and a naive subtraction says they are **763 pixels apart**.

The fix is the *minimum-image convention*: if a difference exceeds half the width, the short way round is the other way. Two comparisons per axis.

The comment names the symptom, which is the useful part: without it, boids near an edge cannot see their neighbours across it, so flocks come apart wherever they touch the seam — and only there, which looks like an edge-case bug in the rendering rather than a distance calculation.

This is the same class of decision as Physarum's wrap-not-clamp: once the world is a torus, **everything that measures distance in it has to know that.**

Alignment and cohesion: two averages, one pass

```
if d2 < neighbor_r * neighbor_r:
    avg_vx += vx_in[unsafe_offset=j]
    avg_vy += vy_in[unsafe_offset=j]
    avg_px += dx
    avg_py += dy
    count += Float32(1.0)
```

Within the neighbour radius, accumulate two things: neighbours' **velocities** and their **relative offsets**. One pass, one counter, and note `d2` is compared against `r²` — no square root in the inner loop, which runs 6.25 million times a frame.

The subtle part is `avg_px += dx` — the *offset*, not the position. So the average that comes out is already relative to this boid:

```
avg_px /= count
new_vx += avg_px * cohesion_w
```

Cohesion is then a single multiply. Accumulating absolute positions would have meant averaging, subtracting this boid's own position, and — much worse — getting the wrap wrong again, because an average of absolute positions across a seam is meaningless. Accumulating already-wrapped offsets sidesteps that entirely.

Alignment is the difference between the neighbours' average velocity and this boid's own:

```
new_vx += (avg_vx - vx) * align_w
```

A steering force *toward* the average heading, not an assignment of it. With `align_w = 1.0` a boid would snap to the flock's heading instantly; at 0.06 it eases in over about twenty frames, and that lag is what makes a turn propagate through a flock as a wave rather than happening everywhere at once.

Separation: an inverse-square push

```
var sep_r = neighbor_r * Float32(0.35)
...
if d2 < sep_r * sep_r and d2 > Float32(0.01):
    var inv = Float32(1.0) / d2
    sep_x -= dx * inv
    sep_y -= dy * inv
```

Three things here.

A smaller radius. Separation acts within 35% of the neighbour radius, so each boid has a large circle it flocks with and a small one it avoids. One parameter, two ranges.

Weighted by 1/d². Not a constant push — the closer a neighbour, the more sharply it is avoided. So distant flockmates barely register while an imminent collision dominates, which is Reynolds' priority ordering falling out of the arithmetic rather than being coded as a priority.

Guarded at d2 > 0.01. Two boids at the same point would divide by zero, and the resulting infinity would propagate into the velocity, then the position, then every neighbour's view of it. One comparison.

Note also that separation is applied **outside** the `if count > 0` block:

```
if count > Float32(0.0):
    ... alignment and cohesion ...
new_vx += sep_x * separation_w
```

Alignment and cohesion need an average, so they need at least one neighbour. Separation is a sum, so it is simply zero when nobody is close. Correct either way, and it avoids a redundant guard.

The mouse, as a fourth force

```
if seek_w > Float32(0.0):
    new_vx += (target_x - x) * seek_w
    new_vy += (target_y - y) * seek_w
```

Held button, and `seek_w` becomes 0.0006. A pull toward the cursor, proportional to distance, added to the same velocity the three rules just modified.

Note what it is *not*: there is no override, no mode, no "seeking" state. The flock still separates, aligns and coheres exactly as before — the seek is one more term in the sum, so what you see is the flock negotiating between the cursor and itself. Release, and it does not need to recover from anything.

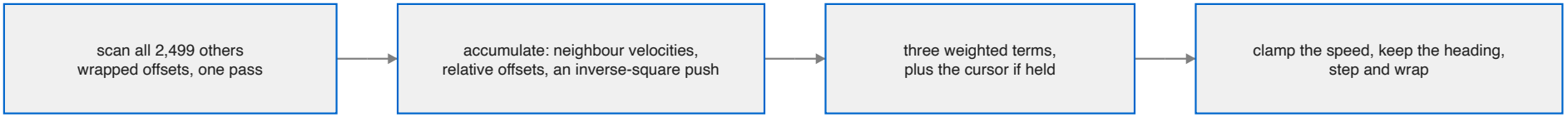
Then the clamps and the step

```
var speed = sqrt(new_vx * new_vx + new_vy * new_vy) + Float32(1e-6)
if speed > MAX_SPEED:
    new_vx = new_vx / speed * MAX_SPEED
elif speed < MIN_SPEED:
    new_vx = new_vx / speed * MIN_SPEED
```

Normalise and rescale — so the clamp changes *speed* while preserving *direction*, which is the whole point. Clamping the components separately would turn a fast diagonal into a different heading.

The `+ 1e-6` guards a boid whose velocity is exactly zero, which would otherwise divide by zero in the very next line.

Then the position update wraps, matching the distance calculation at the top.


scan all 2,499 others
wrapped offsets, one pass
accumulate: neighbour velocities,
relative offsets, an inverse-square push
three weighted terms,
plus the cursor if held
clamp the speed, keep the heading,
step and wrap

3. Why this belongs on a GPU

Boids is the third kernel demo in the collection, and its GPU argument is different from both the others. The difference is worth stating precisely because the commit does:

> *grayscott is a field talking to itself, physarum is agents talking through a shared field, this is agents talking to each other directly, no field between them.*

What direct coupling costs

In Gray-Scott a cell reads nine neighbours at fixed offsets. In Physarum an agent reads three points of a shared map. Neither ever reads another *agent*.

Here, boid *i* reads the position and velocity of all 2,499 others. That has two consequences, and they point in opposite directions.

It is a lot of work. 2,500 × 2,499 = 6.25 million pairs per frame, each with a wrapped-distance calculation and two conditionals.

It is beautiful work for a GPU. Every one of those pairs is independent. There is no ordering, no accumulation shared between threads, no communication. Thread *i* reads the whole input array and writes four values — which is a *gather*, the friendliest shape there is, because gathers never collide.

And the read pattern is ideal in a way stencils are not: **all 2,500 threads walk the same array in the same order at the same time.** At step *j*, every thread in the grid wants `px_in[j]`. That is one broadcast, not 2,500 loads — the best possible case for a cache, and the reason brute force performs far better here than the pair count suggests.

O(n²), on purpose, with numbers

The kernel says so at length, and the honesty is the interesting part:

> *Every boid against every other boid — brute force, on purpose. 2,500 is a visual choice, not a performance ceiling: this GPU held 60fps up to 12,000 boids (144 million pairs a frame) and was still doing 24fps at 20,000 (400 million pairs), so the honest limit is far above what ships here.*

So the shipped number is nearly **five times below** the 60 fps limit, and the reason is not performance:

> *Past a few thousand, the glowing trails stop reading as individual birds and fuse into a continuous flow-field texture — striking in its own right, but no longer showing the thing this demo is FOR: three simple rules, and enough negative space to still see each one's trail and the small flocks it belongs to.*

That is a real result, measured, and then deliberately not shipped. It would have been easy to put 12,000 boids on screen and call it a benchmark.

And the optimisation everybody asks about is named and declined:

> *The interesting, harder trick — bucketing boids into a grid so each one only checks its own neighbourhood — is the right answer for pushing PAST tens of thousands; at a few thousand it is a real optimisation with nothing yet to optimise.*

Worth expanding, because it is the standard next step and it is not free. Spatial binning turns O(n²) into roughly O(n), and it costs:

- a bin-assignment pass, plus a sort or an atomic counter per bin
- **variable work per thread** — a boid in a dense flock checks far more neighbours than one alone, so threads in a warp diverge
- **scattered reads** — neighbours are wherever the bin table points, losing the broadcast pattern above entirely

At 6.25 million perfectly-coalesced, perfectly-uniform pairs, none of that pays. At 400 million it would. The threshold is real and the file says which side of it this is on.

The discipline direct coupling forces

This is the part that would be a silent bug rather than a slow program.

```
def boid_kernel(px_out, py_out, vx_out, vy_out,
                px_in, py_in, vx_in, vy_in, ...):
    """Reads ONLY the `_in` arrays and writes ONLY the `_out` ones -- see the
    file header for why that split, not an in-place update, is the whole
    point: every boid this frame has to see the same frozen instant of
    everyone else, the same discipline `grayscott` uses for its u/v pair."""
```

Update in place and boid 400's *new* position is visible to boid 50's read of it — but only if boid 400's thread happened to run first. From the header:

> *updating in place would let boid 400's new position leak into boid 50's read of it purely by scheduling luck*

The result would still be a flock. It would differ on every run, on every machine, at every occupancy — and there is no test that catches it, because "it looks like birds" is true either way.

This is the third appearance of the same hazard in this tree: Fluid's Jacobi sweep, Gray-Scott's `u`/`v` pair, and now a flock. All three defend structurally — two buffers, swapped — rather than by care. The header calls it exactly that: *"by construction rather than by hope."*

Note it is **four** arrays ping-ponged here, not two, and they must flip together. Position and velocity are both read by every boid, so a frame that saw new positions with old velocities would be just as wrong.

The cheap half

The rendering side is ordinary and worth contrasting.

`decay_kernel` fades the glow buffers, and gets a concession the simulation does not:

> *No neighbour is read here — unlike Physarum's trail, which spreads, a boid's glow only ever fades — so this runs safely in place, with no second buffer needed.*

A pure per-pixel multiply. Each thread touches only its own cell, so there is no old-versus-new question to have. Physarum's trail kernel **blurs**, which reads neighbours, which is exactly why that one needs a second buffer and this one does not.

Two kernels that look similar — decay a buffer — and one needs double buffering because of a neighbour read the other does not make.

Where it sits

example	what couples the work	dispatches/frame

mandelbrot	nothing	1
physarum	a shared map, written racily	3
boids	**every agent to every other, directly**	**4**
grayscott	one cell of diffusion per substep	21
fluid	a global constraint needing a solver	~35

Boids has the strongest coupling of any of them — every pair, every frame — and among the fewest dispatches. Those are not in tension: the coupling is *wide* but it is **not sequential**. All 6.25 million interactions can happen at once.

Fluid's coupling is weaker per-pair and vastly more expensive, because incompressibility must hold everywhere *at once*, and that cannot be done in one pass however many threads you have. Thirty Jacobi sweeps is the price of a constraint that is genuinely global.

Wide coupling is cheap. Deep coupling is not. That is the clearest single thing the five examples together demonstrate.

4. The five kernels

kernel	threads	when
`init_boids`	one per boid	once, and on `r`
`boid`	one per boid	every frame
`splat`	**one per boid**	every frame
`decay`	one per pixel	every frame
`color`	one per pixel	every frame

Note the third row. Every other rendering kernel in this collection is addressed by pixel; this one is not, and the reason is the most interesting decision in the file.

1. `init_boids_kernel`

Physarum's hash, copied — *"example folders are self-contained"* — with the same three-different-multipliers discipline so position and velocity are uncorrelated. No host-side random calls before the first frame.

2. `boid_kernel`

The three rules, covered in chapter 2, and the ping-pong, covered in chapter 3.

One thing about its signature: **sixteen parameters**, eight of them buffers. Four in, four out, plus five weights and a target. The weights are arguments rather than constants so the number keys can switch character live, and the in/out split is four pairs because position and velocity must flip together.

3. `splat_kernel` — the one that runs backwards

```
"""One thread per BOID, not per pixel -- the opposite direction from
every other kernel in this file, because there are 4,000 boids and
589,824 cells, and a boid painting the few pixels around itself is far
cheaper than every pixel asking 4,000 boids whether it is the nearest
one."""
```

This is a **scatter**, and everywhere else in the collection the choice went the other way.

The two options for drawing 2,500 points into 589,824 cells:

	gather (per pixel)	scatter (per boid)
threads	589,824	2,500
work per thread	check all 2,500 boids	write 25 cells
total operations	~1.5 billion	~62,500
writes	each thread owns its cell	threads can collide

Gather is 24,000 times more work, and almost all of it establishes that a pixel has no boid near it. Scatter is obviously right — and it is chosen *despite* being the awkward shape for a GPU, because the arithmetic is not close.

The cost is that scatter has to write where it lands, so two boids painting the same pixel do a racing read-modify-write:

```
gr[unsafe_offset=wi] = gr[unsafe_offset=wi] + r * amt
```

Non-atomic, same as Physarum's deposit — and the same reasoning applies: a lost increment is a fraction of one frame's glow on one pixel, immediately decayed. Physarum's kernel documents its version of this race explicitly; this one does not, which is the one place the file is quieter than its sibling.

Each boid paints a 5 × 5 neighbourhood with a radial falloff:

```
var falloff = Float32(1.0) - d2 / Float32(9.0)
if falloff > Float32(0.0):
```

`d2` is up to 8 at the corners, so `1 - d2/9` makes the corners faint and the extremes of the square drop out entirely — a disc, not a square, from a square loop.

The colour, and a missing `atan2f`

```
var speed = sqrt(bvx * bvx + bvy * bvy) + Float32(1e-6)
var nx = bvx / speed
var ny = bvy / speed
var r = Float32(0.55) + Float32(0.45) * nx
var g = Float32(0.55) + Float32(0.45) * ny
var b = Float32(0.55) - Float32(0.45) * nx
```

> *The colour is the boid's own heading... so two boids painted the same colour are, right now, flying the same way, and a flock turning together turns the SAME colour together. Aligned is not just a rule here; it is what you see.*

That is a genuinely good visualisation decision. Alignment is the rule you cannot see directly — position is obvious, spacing is obvious, but *heading* is invisible in a still image. Mapping it to colour makes the abstract rule the most visible thing on screen.

And the implementation has a story:

> *(Not a hue wheel through atan2: this fork's AIR backend has no atan2f, discovered as a metallib link failure rather than a Mojo-level error — the direction vector's own components make just as good a colour key and need nothing but sqrt.)*

Note **how** it was discovered. Not a compile error, not a type error — an `xcrun metallib` **link** failure, naming a symbol. The Mojo code was well-typed and the function existed in the standard library; there is simply no lowering for it in this backend.

This is the same class of finding as Othello's `llvm.scmp`, where a three-way comparison folded into an intrinsic the Metal backend cannot lower and the kernel failed to link with a message naming an LLVM intrinsic and nothing about Othello. **A link error naming a maths symbol in a GPU build is a lowering gap, not your bug** — and the fix is usually to compute the same thing a different way.

Here the different way is arguably better. `atan2` would give a hue angle, which then needs an HSV-to-RGB conversion; the normalised direction vector's own components *are* two of the three channels. Fewer operations and the same property.

4. `decay_kernel` — and why it needs no second buffer

```
gr[unsafe_offset=idx] = gr[unsafe_offset=idx] * GLOW_DECAY
```

> *No neighbour is read here — unlike Physarum's trail, which spreads, a boid's glow only ever fades — so this runs safely in place, with no second buffer needed.*

Each thread reads and writes only its own cell, so there is no old-versus-new question. Physarum's equivalent kernel **blurs** before it decays, which reads eight neighbours, which is precisely why that one needs double buffering.

Two kernels, both "fade a buffer", and one neighbour read is the whole difference.

Three separate float buffers rather than one, because the glow carries colour — and separate buffers keep each thread's three writes contiguous.

5. `color_kernel`

```
r = r / (Float32(1.0) + r)
```

Fluid's tone curve, per channel:

> *The glow buffers are already coloured — fluid's x/(1+x) tone-map per channel is all that is left, so a dense knot of overlapping boids compresses toward white instead of blowing out to a flat clip.*

Applying it **per channel** rather than to a luminance is what makes a bright knot go white: the channel that saturates first keeps rising more slowly while the others catch up, so overlapping trails desaturate toward white rather than clipping to whichever colour got there first.

That is the fourth appearance of `x/(1+x)` in this collection — Fluid's dye, Physarum's trail, Fernwind's density, and now a flock's glow. Any accumulation with an unbounded range and a bounded display needs one, and this is the cheapest that behaves.

5. A frame, end to end

Four dispatches, and one branch that Gray-Scott went to some trouble to avoid.

Before the loop

```
ctx.enqueue_function(init_kern, px_a, py_a, vx_a, vy_a,
                     grid_dim=(BOID_GRID), block_dim=(BLOCK))
...
ctx.synchronize()
```

Eight boid buffers — two sets of four — plus three glow buffers and the frame. Only the `a` set is initialised; the `b` set is written before it is ever read.

The dispatch sequence

```
if not paused:
    var seek_w = Float32(0.0006) if g_mouse_down()[] != 0 else Float32(0.0)
    if parity == 0:
        ctx.enqueue_function(
            boid_kern, px_b, py_b, vx_b, vy_b, px_a, py_a, vx_a, vy_a, ...)
        ctx.enqueue_function(
            splat_kern, gr, gg, gb, px_b, py_b, vx_b, vy_b, ...)
    else:
        ctx.enqueue_function(
            boid_kern, px_a, py_a, vx_a, vy_a, px_b, py_b, vx_b, vy_b, ...)
        ctx.enqueue_function(
            splat_kern, gr, gg, gb, px_a, py_a, vx_a, vy_a, ...)
    ctx.enqueue_function(decay_kern, gr, gg, gb, ...)
    parity = 1 - parity
```

```
ctx.enqueue_function(color_kern, dev, gr, gg, gb, ...)
ctx.synchronize()
```

Read the order:

1. **boids** — read one set, write the other
2. **splat** — paint from the set just written
3. **decay** — fade the glow
4. **colour** — tone-map into BGRA

Note **splat comes before decay**. So a boid's contribution this frame is painted at full brightness and *then* faded along with everything else, which means the newest position is exactly one decay step brighter than the previous one — a smooth comet tail rather than a hard bright head.

Note also that `color_kern` sits outside the `if not paused` block, so a paused frame is still drawn. Same as Physarum.

The parity, spelled out

Physarum tracked parity with a variable and a conditional buffer pick:

```
parity = 1 - parity
cur = trail_a if parity == 0 else trail_b
```

Boids cannot do that, because it has **four** buffers to flip, not one — and they have to flip together. Selecting four pointers with four conditionals would be worse to read than the explicit branch, so the file writes both arms out:

```
if parity == 0:
    ... boid_kern(b ← a); splat_kern(from b) ...
else:
    ... boid_kern(a ← b); splat_kern(from a) ...
```

Duplicated, and correct in a way that is checkable by eye: each arm names its own source and destination, and the splat in each arm reads the set that arm just wrote. A pointer-swapping version would put that correspondence one level of indirection away.

Three ping-pong strategies in three sibling examples, each fitting its own shape:

	how	why
Gray-Scott	even substep count, argument order	20 steps per frame, so it can end where it started
Physarum	a parity variable, one buffer picked	one diffusion per frame, one buffer to track
Boids	**an explicit branch, both arms written**	**four buffers, and they must flip together**

The mouse as a weight, not a mode

```
var seek_w = Float32(0.0006) if g_mouse_down()[] != 0 else Float32(0.0)
```

Held button, and the seek weight is non-zero; released, and it is zero. There is no state, no transition, no easing — the term is simply present or absent in a sum.

That is the same design as Physarum's attract kernel, which forces nothing and merely gives agents something bright to find. Both interactions cost almost nothing because they reuse the mechanism that was already the whole model.

Preset switching

Four floats change and the next dispatch is a different creature — the boids are **not** reset and their positions are kept. So an established flock reorganises: switch from `flock` to `scatter` and you watch the groups dissolve rather than seeing a fresh random field.

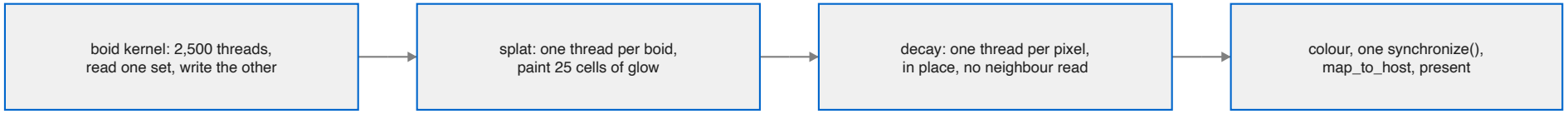
And the commit says that is exactly how it was verified:

> *the default preset shows distinct swirling flocks with individually legible trails: switching to "scatter" (weak alignment and cohesion, strong separation) produces visibly correct chaos instead — no large flocks at all, every bird darting on its own — proof the three rules are actually reaching the kernel differently, not a palette change wearing a different name.*

The specific failure being ruled out is worth naming, because it is the same one Gray-Scott and Physarum tested for: a preset switch that updates the local variables and the window title but fails to pass them into the dispatch looks entirely correct from outside. Every automated check passes. Only two visibly different behaviours rule it out.

Then the usual tail

`ctx.synchronize()` — the one blocking point — then the copy to the host buffer, the optional PNG from exactly the bytes about to be presented, and `replaceRegion:` onto the drawable. Identical to Gray-Scott and Physarum, including the comment about why the snapshot is taken there and not later.



6. What to understand

Eight things about this example are not obvious from watching it.

1. Direct coupling forces the ping-pong

A boid reads every *other* boid. Update in place and boid 400's new position leaks into boid 50's read of it — but only if 400's thread ran first.

> *purely by scheduling luck*

The result is still a flock. It differs on every run, on every machine, at every occupancy, and **no test catches it**, because "it looks like birds" is true either way.

Four buffers, flipped together, so every boid sees the same frozen instant: *"by construction rather than by hope."* This is the third appearance of the hazard in this tree, after Fluid's Jacobi sweep and Gray-Scott's `u`/`v` pair.

2. Brute force is a measured choice, not a shortcut

6.25 million pairs a frame, and the numbers were taken before the count was picked:

boids	pairs/frame	result
2,500	6.25 M	ships
12,000	144 M	60 fps
20,000	400 M	24 fps

2,500 is nearly five times *below* the 60 fps limit, and the reason is editorial:

> *Past a few thousand, the glowing trails stop reading as individual birds and fuse into a continuous flow-field texture — striking in its own right, but no longer showing the thing this demo is FOR.*

3. Spatial binning is the right answer, later

Bucketing boids into a grid turns O(n²) into roughly O(n) — and costs a bin-assignment pass, **variable work per thread** (dense flocks check more neighbours, so warps diverge), and **scattered reads** in place of the current broadcast pattern.

> *at a few thousand it is a real optimisation with nothing yet to optimise*

4. Brute force is unusually cache-friendly

At step *j*, every one of the 2,500 threads wants `px_in[j]` — the same address. That is a broadcast, not 2,500 loads, and it is why brute force runs better than the pair count suggests. Binning would destroy exactly this.

5. A torus has to be a torus everywhere

```
if dx > Float32(WIDTH) * Float32(0.5):
    dx -= Float32(WIDTH)
```

Two boids across the seam are at x = 766 and x = 3; naive subtraction says 763 apart. Without the minimum-image fix, flocks come apart at the edges **and only there**, which reads as a rendering bug.

Same class as Physarum's wrap-not-clamp: once the world wraps, everything measuring distance in it must know.

6. Cohesion accumulates offsets, not positions

```
avg_px += dx        # the already-wrapped offset
```

So the average is relative and cohesion is one multiply. Averaging *absolute* positions would need a subtraction afterwards and — much worse — would be meaningless across a seam, since the mean of x = 766 and x = 3 is the middle of the screen.

7. A missing `atan2f`, found at link time

The colour was going to be a hue wheel through `atan2`. This fork's AIR backend has no `atan2f`, and it surfaced as an **xcrun metallib link failure** — not a compile error, not a type error. The Mojo was well-typed and the function existed; there is no lowering for it.

Same class as Othello's `llvm.scmp`. **A link error naming a maths symbol in a GPU build is a lowering gap, not your bug.**

The replacement is arguably better: the normalised velocity's own components *are* two channels, where `atan2` would give an angle still needing an HSV conversion.

8. Heading is mapped to colour, and that is the visualisation

Position and spacing are visible in a still frame. **Heading is not.** Colouring by direction makes alignment — the one rule you could not otherwise see — the most obvious thing on screen:

> *a flock turning together turns the SAME colour together. Aligned is not just a rule here; it is what you see.*

Things that will bite: the model

If you change…	…this happens
the minimum-image wrap	flocks tear apart at the seams, and only there
cohesion to average absolute positions	it is meaningless across a seam
the `d2 > 0.01` separation guard	two coincident boids divide by zero and poison their neighbours
the speed clamp to per-component	a fast diagonal silently changes heading
`MIN_SPEED` to zero	a boid whose forces cancel stops dead and becomes an obstacle
alignment weight toward 1.0	boids snap to the flock heading; turns stop propagating as waves

Things that will bite: the machinery

If you change…	…this happens
the boid kernel to update in place	nondeterministic flocking, no error, no failing test
the four buffers to flip separately	new positions paired with old velocities
`splat` to one thread per pixel	~1.5 billion checks instead of 62,500 writes
`decay` to read a neighbour	it stops being safe in place and needs a second buffer
the tone map to luminance instead of per channel	bright knots clip to a colour instead of going white
`atan2` back into the colour	the metallib link fails, naming a symbol

Controls

hold	the flock steers toward the cursor
1–4	flock, swarm, school, scatter — live, positions kept
r / **space** / **s** / **q**	reseed · pause · save a PNG · quit
`BOIDS_FRAMES=N`	render N frames unfocused, then exit

And beside the others

example	what couples the work	dispatches/frame
mandelbrot	nothing	1
physarum	a shared map, written racily	3
boids	**every agent to every other, directly**	**4**
grayscott	one cell of diffusion per substep	21
fluid	a global constraint needing a solver	~35

Boids has the strongest coupling here and among the fewest dispatches, and those are not in tension: its coupling is **wide** but not **sequential** — all 6.25 million interactions happen at once. Fluid's is weaker per pair and far more expensive, because incompressibility must hold everywhere simultaneously and no number of threads collapses that into one pass.

Wide coupling is cheap. Deep coupling is not.